DESENVOLVIMENTO ASSISTIDO POR IA · BLOCO 4

# Legado: leitura, navegação e mudança segura

4

**Sexta, 23/10/2026 · 19h às 22h**

Ferramenta: Claude Code · modo manual por padrão

Pós-Graduação em Engenharia de Software e IA · CESAR School

CESAR

ABERTURA

# Vocabulário do bloco

| | |
|---|---|
| **legacy code** | código sem teste, difícil de mudar com segurança |
| **characterization test** | teste que registra o comportamento atual |
| **seam (costura)** | ponto onde dá para trocar o comportamento sem editar ali |
| **golden master** | saída gravada usada como referência |
| **codemod** | transformação de código automatizada |

| | |
|---|---|
| **AST** | árvore de sintaxe abstrata: o código como estrutura |
| **strangler fig** | figueira estranguladora: substituir aos poucos, por fora |
| **feature flag** | chave que liga ou desliga um caminho |
| **blast radius** | raio de impacto de uma mudança |
| **Chesterton's fence** | descubra por que existe antes de remover |

# Qual parte do código da sua empresa ninguém ousa mexer? Por quê?

Responda no chat

ABERTURA

# Agenda do bloco

## Hoje · 23/10

19h00
**Exposição: hipótese e fato, leitura, verificação, rede de segurança**

19h55
**Pausa**

**20h05 às 21h25**
**AV1.4 · ler legado com prova em um repositório que vocês não conhecem**

21h25
**Socialização das respostas erradas e fechamento**

## Disciplina · 6 blocos

B1 · 16/10
**Modos e ambiente**

B2 · 17/10
**Escopo e refatoração**

B3 · 17/10
**Testes e docs**

B4 · agora
**Legado**

B5 · 24/10
**Revisão e segurança**

B6 · 24/10
**Produtividade e time**

HIPÓTESE E FATO

# Legado é código sem teste, não código velho

Código que ninguém entende bem. Antes de remover algo estranho, descubra por que existe (cerca de Chesterton).





C E S A R

HIPÓTESE E FATO

# A IA produz hipóteses. O código e a execução produzem fatos.

HIPÓTESE

## O que a IA responde

Acelera a orientação.

FATO

## O que o código e a execução mostram

Você prova cada afirmação.

LEITURA ASSISTIDA

# Duas direções de leitura

DE CIMA PARA BAIXO

## Do sistema para o detalhe

README, build, pontos de entrada, configuração por ambiente.

DE BAIXO PARA CIMA

## Dos dados para o sistema

Modelo de dados, entidades, quem as manipula.

LEITURA ASSISTIDA

# Técnicas que funcionam com ou sem IA

1. Fatia vertical: siga um caso de uso até a persistência
2. Arqueologia no git
3. Pontos quentes (muda muito e é complexo)
4. Testes como documentação
5. Mapa e depois zoom

LEITURA ASSISTIDA

# Arqueologia no histórico

Comandos de git que revelam o porquê.

```
git log --stat -- caminho/do/arquivo     # histórico e tamanho das mudanças
git blame -w caminho/do/arquivo          # quem mexeu em cada linha
git log -S"termo" --oneline              # quando um termo apareceu ou sumiu
git shortlog -sn -- caminho/             # quem conhece esta área
```

LEITURA ASSISTIDA

# Três papéis do agente na leitura

## Buscador



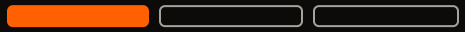
Confie mais

## Sumarizador



Confie menos

## Hipotetizador



Verifique sempre

**CONFIANÇA** diminui da esquerda para a direita; a verificação aumenta.

LEITURA ASSISTIDA

# Pergunta fraca x pergunta que força evidência

RUIM **"Como funciona o checkout?"**

BOM

```
Pergunta: Quais componentes escrevem na tabela `pedidos`?
Regras de resposta:
1. Liste cada ponto de escrita com caminho:linha.
2. Informe quais arquivos você abriu e quais NÃO abriu.
3. Separe: (a) confirmado no código, (b) inferido, (c) não sei.
4. Diga como eu poderia refutar sua resposta (qual busca eu faria).
```

VERIFICAÇÃO

# Segunda opinião barata: ferramentas determinísticas

| Necessidade | Ferramenta |
|---|---|
| **Quem referencia este símbolo** | LSP, ripgrep |
| **Grafo de dependências** | madge, import-linter, jdeps |
| **Consulta estrutural** | ast-grep, Semgrep, CodeQL |
| **O que executa de fato** | cobertura em execução, depurador, logs |
| **Origem de uma linha** | `git blame`, `git log -S` |

VERIFICAÇÃO

# Protocolo afirmação → prova (leitura orientada pelo ceticismo)

1. Registre a afirmação em uma frase testável
2. Busque evidência a favor
3. Busque contraevidência
4. Confirme por execução
5. Só então aja

VERIFICAÇÃO

# Onde a resposta plausível erra (1/2)

| Fonte do erro | Exemplo |
|---|---|
| **Despacho dinâmico e reflexão** | Método "sem chamadas" é chamado por nome |
| **Injeção de dependência** | A implementação real não é a que parece |
| **Feature flags e config por ambiente** | Caminho morto ativo só em produção |
| **Código gerado e monkey patching** | O que executa não está no arquivo que você lê |

VERIFICAÇÃO

# Onde a resposta plausível erra (2/2)

| Fonte do erro | Exemplo |
|---|---|
| **Escritas laterais** | Jobs, scripts e migrations também escrevem na tabela |
| **Nomes enganosos** | `validate_user` só loga |
| **Comentários desatualizados** | Repetidos como fato |
| **Consumidores externos** | O endpoint "sem uso" tem quem chame |

TESTE DE REFUTAÇÃO

# Para toda afirmação "só X faz Y" ou "nunca é chamado": formule a busca que a derrubaria.

Se não consegue formular, não está pronto para agir.

# Rede de segurança antes de mudar

1. Teste de caracterização e golden master
2. Costuras (seams)
3. Gravar e reproduzir (record/replay)
4. Execução em paralelo (sombra)
5. Figueira estranguladora
6. Feature flag

**REGRA** Quanto menos teste, menor o passo.

REDE DE SEGURANÇA

# Sem teste e sem costura, o primeiro entregável não é a mudança: é a rede

Peça à IA testes de caracterização primeiro; mude depois.

MIGRAÇÃO EM ESCALA

# Migração em escala no Google

**~80%**

das modificações nas mudanças submetidas foram produzidas pela IA (migração de IDs de 32 para 64 bits)

**~50%**

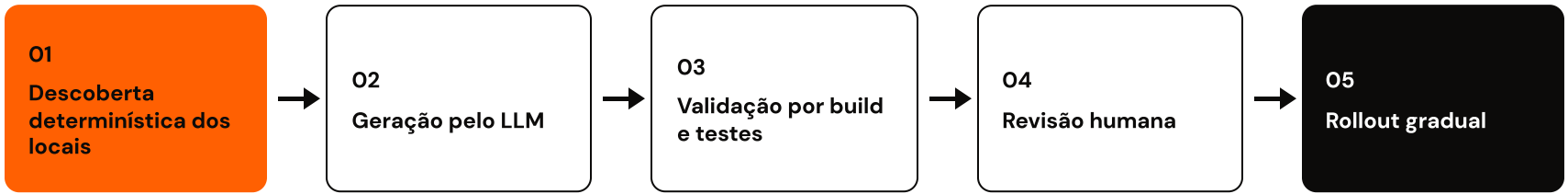
de redução estimada do tempo total da migração

**~74%**

de 93.574 edições geradas por LLM, em estudo posterior: 39 migrações em 12 meses, 595 mudanças

**ESTIMATIVA DOS PRÓPRIOS ENGENHEIROS** Tipo de evidência: relato de experiência.

# O que torna isso possível: um pipeline, não só um modelo



MIGRAÇÃO EM ESCALA

# Codemod, IA ou humano?

| Tipo de mudança | Quem faz |
|---|---|
| **Transformação sintática regular** | Codemod/AST: jscodeshift, OpenRewrite, libcst, ast-grep |
| **Casos irregulares ou residuais** | LLM em lotes pequenos |
| **Decisão de semântica (timezone, arredondamento, nulos)** | Humano |

# O que sempre precisa de humano no meio

1. Diferenças semânticas entre bibliotecas
2. API pública consumida por terceiros
3. Migração de dados
4. Concorrência
5. Segurança

RISCO

**Risco ≈** Raio de impacto × Incerteza de comportamento × Ausência de feedback

RISCO

# Como reduzir o risco

1. Congele o comportamento
2. Fatie: uma fatia por PR
3. Compile e cheque tipos a cada fatia
4. Defina flag e rollback antes
5. Use canário
6. Pare e reavalie se o primeiro lote surpreender

RISCO

# Registre o que aprendeu e rotule o que não foi verificado

Documento de arquitetura viva no repositório. Cada item marcado:

**VERIFICADO (COM TRECHO)** A afirmação e o `arquivo:linha` que a prova

**HIPÓTESE** A afirmação ainda sem prova

**Assim, resumo de IA não vira documentação oficial.**

**Janela: 20h05 às 21h25** Tempo encerrado

# Ler legado com prova

1. Cada grupo responde **5 perguntas de arquitetura** com apoio do agente e **prova cada resposta apontando o trecho**
2. Registre uma resposta que parecia certa e estava errada
3. Grave a saída de dois relatórios antes de mudar
4. Escreva um plano de migração em fatias

**Tempo em salas:** 50 min

**ONDE**

`exercicios/aula-4/README.md`

`exercicios/aula-4/template-registro.md`

Código em `caso/legado/`, liberado agora pelo professor

**NÃO** **leia o README do repositório antes**

# AV1.4: formato da resposta

| Afirmação | Trecho (arquivo:linha) | Contraevidência buscada | Status |
|---|---|---|---|
| **Só o checkout escreve em `pedidos`** | `app/checkout.py:88` | `rg "INSERT INTO pedidos"` achou um job noturno | Refutada |

Exemplo fictício. Status: confirmada, refutada ou incerta.

## Plano de migração

| Fatia | Mudança | Mecânica ou decisão humana | Verificação |
|---|---|---|---|

# Exemplos de buscas para refutar

Buscas por escritas na tabela, disparos de jobs e leitura de variáveis de ambiente.

PROJETO FICTÍCIO

```
rg -n "INSERT INTO pedidos|UPDATE pedidos|Pedido.objects.(create|update)"
rg -n "delay\(|apply_async\(|crontab"
rg -n "os\.environ|getenv"
```

# AV1.4 em salas: 50 minutos

| | |
|---|---|
| Grupos | **4 a 5 pessoas** |
| Perguntas | **divididas, provas cruzadas** |
| Transição de salas | **5 a 8 min** |

VOLTEM ÀS

**21h25**

Tempo encerrado

# Quais respostas estavam erradas com aparência de certas?

**3 a 5 grupos apresentam a refutação**

EXEMPLO

# Exemplo: migrar um gateway de pagamentos

TRADUZA TUDO

## Pedir a tradução total de uma vez

CAMINHO SEGURO

1. Isole os pontos de entrada
2. Caracterize com respostas gravadas
3. Isole por adaptador
4. Estrangule por tráfego com flag
5. Execução em paralelo e conciliação diária
6. Reversão definida
7. Humano em idempotência, webhooks, estornos, arredondamento e PCI

FECHAMENTO

# Em legado, a IA acelera a leitura e não substitui a conferência

Prove cada afirmação.

Caracterize antes de mudar.

Mude em fatias pequenas.

FECHAMENTO

# Referências e próximo bloco

**Nikolov et al. (2025).** Migrações em escala no Google.
arXiv 2501.06972

**Feathers.** *Working Effectively with Legacy Code* [consolidado].

AMANHÃ · 24/10, 9H

## Revisão, segurança e dívida do código assistido

**AV1.5** às 10h10

github.com/jamissonfreitas/repo_alunos_d12_esia



CESAR